

Phases 2, 3 & 4 — Résolution & Deep Learning

Fiches tâches détaillées · Organisation parallèle P2 ■ P4 · Répartition 4 personnes

Organisation des phases

Les phases 2 et 4 démarrent en parallèle dès que le générateur (T7, phase 1) est validé. La phase 3 démarre après la phase 2 — elle réutilise NNH comme solution initiale du recuit. La phase 5 (étude expérimentale) attend la fin des trois méthodes.

Phase 2

Dépend de T7 (générateur). Démarre dès que T7 est stable.

Phase 3

Dépend de Phase 2 terminée. NNH sert de solution initiale au recuit.

Phase 4

Dépend de T7 uniquement. Parallèle à P2 et P3. Pas de dépendance aux heuristiques.

Phase 5

Dépend de P2 + P3 + P4 terminées. Besoin des 3 méthodes pour comparer.

2

Méthode 1 — NNH + 2-opt

Baseline heuristique · Livrable final
partie 2

Livrable final

T1
0

Implémenter Nearest Neighbor Heuristic (NNH)

code

heuristique

POURQUOI

NNH est la méthode de référence la plus simple pour le TSP. Elle sert de baseline — toute méthode suivante doit faire mieux. Sa simplicité permet de valider rapidement que le générateur d'instances fonctionne correctement.

COMMENT

Depuis chaque ville possible comme point de départ, appliquer la règle glouton : aller toujours à la ville non visitée la plus proche. Garder la meilleure tournée sur tous les départs. Implémenter avec numpy pour les calculs vectorisés sur la matrice de distances.

```
def nnh(dist_matrix, start=0):  
    n = len(dist_matrix)  
    visited = [False] * n  
    tour = [start]; visited[start] = True  
    for _ in range(n-1):  
        current = tour[-1]  
        # choisir la ville non visitée la plus proche  
        next_city = min([j for j in range(n) if not visited[j]],  
                        key=lambda j: dist_matrix[current][j])  
        tour.append(next_city); visited[next_city] = True  
    return tour # retour depot implicite
```

Tester sur une instance de $n=5$ dont on connaît la solution optimale pour valider.

NOTIONS ABORDÉES

Algorithme glouton

$O(n^2)$ par départ

numpy vectorisé

Baseline qualité

Charge : 1 personne · ~1h30

Livrable : Section "Méthode 1" du notebook final

T1 1

Implémenter l'amélioration locale 2-opt

[code](#)[amélioration locale](#)

POURQUOI

NNH seul donne ~20-25% sous-optimal. Le 2-opt améliore systématiquement la solution en éliminant les croisements d'arêtes. NNH + 2-opt est la combinaison standard qui tombe à ~5-10% sous-optimal et sert de vraie référence qualité.

COMMENT

Pour chaque paire d'arêtes $(i, i+1)$ et $(j, j+1)$ de la tournée : calculer si inverser le segment entre $i+1$ et j réduit le coût total. Si oui, accepter. Répéter jusqu'à ce qu'aucun échange ne soit bénéfique (minimum local atteint).

```
def two_opt(tour, dist_matrix):
    improved = True
    while improved:
        improved = False
        for i in range(1, len(tour)-2):
            for j in range(i+1, len(tour)):
                delta = (dist_matrix[tour[i-1]][tour[j]]
                        + dist_matrix[tour[i]][tour[j+1 % len(tour)]]
                        - dist_matrix[tour[i-1]][tour[i]]
                        - dist_matrix[tour[j]][tour[j+1 % len(tour)]])
                if delta < -1e-10: # amelioration
                    tour[i:j+1] = tour[i:j+1][::-1]
                    improved = True
    return tour
```

NOTIONS ABORDÉES

[Amélioration locale](#)[Minimum local](#) [\$O\(n^2\)\$ par itération](#)[Inversion de segment](#)

Charge : 1 personne · ~1h30

Livrable : Section "Méthode 1" du notebook final

T1 2

Intégrer les contraintes VRPTW dans NNH

code

contraintes

POURQUOI

NNH de base ignore les fenêtres temporelles et la capacité. Il faut adapter le choix du prochain client pour respecter ces contraintes, ou les gérer via un système de pénalités (soft constraints) qui permettra la comparaison directe avec le recuit simulé.

COMMENT

À chaque étape de NNH, filtrer les villes candidates selon : (1) capacité restante $\geq$ demande du client, (2) arrivée possible dans la fenêtre $[a_i, b_i]$ en tenant compte du temps de trajet + service. Si aucun client n'est atteignable sans violation : ajouter pénalité ou démarrer une nouvelle tournée (cas multi-véhicules).

Vérification avant chaque visite :

```
arrivee = max(t_current + c_ij, a_j) # attente si arrivée trop tôt
if arrivee > b_j: invalide (hard) ou penalite += lambda*(arrivee - b_j)
if charge + q_j > Q: invalide (hard) ou nouveau vehicule
```

Coût total avec pénalité soft :

```
cout = SUM(c_ij) + lambda * SUM(max(0, t_ik - b_i))
```

Choisir dès maintenant hard ou soft et documenter ce choix — le recuit simulé (T16) doit faire le même choix pour que la comparaison soit loyale.

NOTIONS ABORDÉES

Fenêtres temporelles

Capacité multi-véhicules

Hard vs soft constraints

Propagation du temps

Charge : 1 personne · ~2h

Livrable : Section "Méthode 1" du notebook final

T1
3

Logger solution, coût et temps de calcul

code

mesure

POURQUOI

Le format de logging doit être identique pour toutes les méthodes. C'est ce qui permettra la comparaison directe dans la phase 5. Un format incohérent entre méthodes rend l'étude expérimentale impossible.

COMMENT

Créer une fonction standardisée qui retourne un dictionnaire de résultats pour n'importe quelle méthode. Mesurer le temps avec `time.perf_counter()` (plus précis que `time.time()`). Stocker aussi la seed pour reproductibilité.

```
result = {
    'method': 'NNH+2opt',
    'instance_seed': seed,
    'n': n,
    'tour': [0, 3, 1, 4, 2, 0], # liste des indices
    'total_cost': 142.7,
    'n_vehicles': 2,
    'violations': 0, # nb de contraintes violées
    'time_ms': 3.4, # temps calcul en ms
    'gap_vs_optimal': None, # rempli si optimal connu
}
```

NOTIONS ABORDÉES

`time.perf_counter()`

Format JSON
standardisé

Reproductibilité seed

Métriques comparables

Charge : Peut être fait en parallèle de T12 · ~1h

Livable : Réutilisé par T18, T24, T26, T27 (toutes phases)

T1
4

Valider sur petites instances vs solution exacte

validation

livrable

POURQUOI

Sans validation contre une solution optimale connue, impossible de savoir si l'implémentation est correcte. La comparaison sur $n \leq 12$ permet de calculer le gap réel de NNH+2opt et de détecter les bugs avant d'aller sur de grandes instances.

COMMENT

Implémenter un solveur exact par force brute (toutes les permutations) sur $n \leq 12$. Comparer NNH+2opt vs optimal sur 30 instances aléatoires. Calculer le gap moyen (% au-dessus de l'optimal). Résultat attendu : ~20-25% sans 2-opt, ~5-10% avec 2-opt.

```
from itertools import permutations

def exact_brute_force(dist_matrix):
    n = len(dist_matrix)
    best_cost = float("inf"); best_tour = None
    for perm in permutations(range(1, n)):
        tour = [0] + list(perm) + [0]
        cost = sum(dist_matrix[tour[i]][tour[i+1]]
                    for i in range(len(tour)-1))
        if cost < best_cost: best_cost = cost; best_tour = tour
    return best_tour, best_cost

gap = (nnh_cost - optimal_cost) / optimal_cost * 100
```

NOTIONS ABORDÉES

Vérification
implémentation

Gap % vs optimal

itertools.permutations

Statistiques sur 30
instances

Charge : 1 personne · ~1h30 — validation critique

Livrable : Section "Validation méthode 1" du notebook final

Méthode 2 — Recuit

3 Simulé

Métaheuristique · Démarre après
Phase 2 · Livrable final partie 2

Livrable final

Pourquoi séquentiel après P2 ? Le recuit simulé a besoin d'une solution initiale. Utiliser NNH+2opt comme point de départ accélère la convergence et donne un meilleur résultat final que de partir d'une tournée aléatoire. C'est aussi une décision de conception à documenter dans le notebook.

T1
5

Implémenter le Recuit Simulé (SA)

code

métaheuristique

POURQUOI

Le recuit simulé est la meilleure méthode classique pour le TSP. Il s'échappe des minima locaux grâce à son critère d'acceptation probabiliste. C'est la référence de performance que le Deep Learning devra approcher ou dépasser.

COMMENT

Partir de la solution NNH+2opt. À chaque itération : générer une solution voisine par perturbation 2-opt aléatoire. Calculer $\delta = \text{nouveau_coût} - \text{coût_actuel}$. Si $\delta < 0$: accepter toujours. Si $\delta \geq 0$: accepter avec probabilité $e^{(-\delta/T)}$. Diminuer T selon alpha après chaque palier.

```
def simulated_annealing(tour, dist, T0, alpha, n_iter):
    current = tour.copy(); best = tour.copy()
    cost = tour_cost(current, dist)
    best_cost = cost; T = T0
    for _ in range(n_iter):
        # perturbation 2-opt aleatoire
        i, j = sorted(random.sample(range(1, len(tour)), 2))
        neighbor = current[:i] + current[i:j+1][::-1] + current[j+1:]
        new_cost = tour_cost(neighbor, dist)
        delta = new_cost - cost
        if delta < 0 or random.random() < exp(-delta / T):
            current = neighbor; cost = new_cost
        if cost < best_cost: best = current.copy(); best_cost = cost
        T *= alpha # refroidissement
    return best, best_cost
```

NOTIONS ABORDÉES

Critère d'acceptation
 $e^{(-\Delta C/T)}$

Programme de
refroidissement

Perturbation 2-opt

Métaheuristique

Charge : 1 personne · ~2h

Livrable : Section "Méthode 2" du notebook final

T1
6

Intégrer les contraintes via pénalités (soft)

code

contraintes soft

POURQUOI

Le recuit simulé explore des solutions infaisables temporairement — c'est même utile pour traverser des régions difficiles. Les contraintes soft (pénalités) permettent cela contrairement aux hard constraints qui rejetteraient trop de solutions candidates.

COMMENT

Ajouter un terme de pénalité dans la fonction de coût. Le paramètre lambda contrôle le poids des violations. Un lambda trop faible ignore les contraintes, trop fort les rend bloquantes. Doit être cohérent avec le choix fait en T12.

```
def penalized_cost(tour, dist, time_windows, demands, Q, lam=100):
    cost = tour_cost(tour, dist)
    # penalite fenetres temporelles
    t = 0
    for i in range(len(tour)-1):
        t = max(t + dist[tour[i]][tour[i+1]], time_windows[tour[i+1]][0])
        cost += lam * max(0, t - time_windows[tour[i+1]][1])
    # penalite capacite
    load = sum(demands[c] for c in tour)
    cost += lam * max(0, load - Q)
    return cost
```

Le lambda doit être calibré empiriquement. Tester lambda = 10, 100, 1000 et observer l'impact sur la qualité et le nombre de violations en fin de run.

NOTIONS ABORDÉES

Soft constraints

Paramètre lambda

Propagation temporelle

Cohérence avec T12

Charge : 1 personne · ~1h30

Livrable : Section "Méthode 2" du notebook final

T1
7

Étude des hyperparamètres T_0 , α , n_{iter}

expérimental

livrable

POURQUOI

Les hyperparamètres du recuit simulé ont un impact majeur sur la qualité. Un T_0 trop bas piège dans le minimum local de départ. Un α trop proche de 1 refroidit trop lentement (lent mais bon). Cette étude est un argument fort dans l'étude expérimentale.

COMMENT

Grid search léger sur une instance de $n=50$: T_0 in $[10, 100, 1000]$, α in $[0.90, 0.95, 0.99]$, n_{iter} in $[1000, 5000, 10000]$. Pour chaque combinaison : 5 runs, mesurer coût moyen, écart-type, temps. Tracer la courbe coût vs itérations pour les meilleurs hyperparamètres.

Grille : $3 \times 3 \times 3 = 27$ combinaisons $\times 5$ runs = 135 executions

Critères de sélection :

- Meilleur compromis qualite/temps
- Faible ecart-type (stabilite)
- Convergence visible sur la courbe

Valeurs typiques litterature TSP :

$T_0 = \text{distance moyenne} \times 0.5$

$\alpha = 0.995$

$n_{\text{iter}} = n^2 \times 100$

NOTIONS ABORDÉES

Grid search

Courbe de convergence

Écart-type

Compromis
qualité/temps

Charge : 1 personne · ~2h

Livrable : Section "Étude hyperparamètres" du notebook final

T1
8

Comparer SA vs NNH+2-opt sur mêmes instances

comparaison

livrable

POURQUOI

La comparaison entre les deux méthodes classiques est le cœur de la partie 2 du livrable. Elle établit la hiérarchie de qualité et prépare la comparaison avec le Deep Learning en phase 5.

COMMENT

Sur les mêmes instances (même seed, même n), lancer NNH+2opt et SA et comparer : coût final, gap vs optimal (pour petits n), temps de calcul, stabilité (écart-type sur 10 runs). Tracer : courbe qualité vs n, courbe temps vs n pour les deux méthodes.

Pour chaque taille n in {10, 20, 50, 100} :

- Générer 30 instances (seeds 0..29)
- Lancer NNH+2opt et SA sur chaque instance
- Calculer : coût moyen, écart-type, temps moyen
- Calculer gap : $(SA_cost - NNH_cost) / NNH_cost \times 100$

Résultat attendu :

SA ~5-15% mieux que NNH+2opt sur qualité

SA ~10-50x plus lent que NNH+2opt sur temps

NOTIONS ABORDÉES

Protocole expérimental

Même seed = équité

matplotlib comparaison

Statistiques sur 30 runs

Charge : 1 personne · ~2h

Livrable : Section "Comparaison P2 vs P3" du notebook final

4 Méthode 3 — Deep Learning (RL)

Reinforcement Learning · En parallèle
de P2+P3 · Démarre tôt

Commencer tôt !

Point d'attention critique. Le RL prend du temps à converger et à déboguer. Il faut démarrer cette phase dès que T7 (générateur) est stable — pas en dernière semaine. L'objectif n'est pas d'égaliser Kool 2019 mais de montrer qu'un agent apprend à construire des tournées et de mesurer son temps d'inférence constant.

T1
9

Définir l'environnement RL

DL

modélisation

POURQUOI

L'environnement est l'interface entre l'agent et le problème. Un environnement mal défini (mauvaise représentation de l'état, récompense mal calibrée) empêche l'agent d'apprendre. C'est la décision de conception la plus importante de la phase 4.

COMMENT

Définir formellement les trois composants de l'environnement RL. L'état doit encoder toute l'information nécessaire pour prendre la prochaine décision. La récompense doit être dense (donnée à chaque étape) et non seulement terminale, sinon l'apprentissage est trop lent.

État (state) à chaque étape t :

- Villes restantes à visiter (masque binaire)
- Position courante du véhicule (indice)
- Heure courante t_{current}
- Capacité restante
- Coordonnées de toutes les villes (contexte global)

Action : indice de la prochaine ville à visiter
(seules les villes non visitées et accessibles sont valides)

Récompense : $-c_{ij}$ (coût du déplacement)
+ pénalité si violation time window ou capacité

NOTIONS ABORDÉES

MDP (Markov Decision Process)

Représentation d'état

Masque d'actions valides

Récompense dense

gymnasium API

Charge : 1 personne · ~2h

Livrable : Section "Environnement RL" du notebook final

T2
0

Implémenter la boucle REINFORCE

DL

RL

POURQUOI

REINFORCE (Williams 1992) est l'algorithme de policy gradient le plus simple. Il entraîne le modèle à augmenter la probabilité des actions qui ont mené à de bonnes récompenses. C'est le point de départ avant tout modèle complexe.

COMMENT

À chaque épisode : laisser l'agent construire une tournée complète en échantillonnant ses actions. Calculer le retour G (somme des récompenses). Mettre à jour le réseau par descente de gradient en maximisant $E[G * \log \pi(\text{actions})]$. Utiliser une baseline (coût moyen récent) pour réduire la variance.

```
for episode in range(n_episodes):
    # generer une tournée complete
    states, actions, rewards = rollout(model, instance)
    G = sum(rewards) # retour total
    # baseline = moyenne mobile des retours recents
    advantage = G - baseline.update(G)
    # calcul du gradient REINFORCE
    log_probs = compute_log_probs(states, actions, model)
    loss = -advantage * sum(log_probs)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Déboguer d'abord sur $n=5$ sans contraintes. Vérifier que la récompense augmente avec les épisodes avant d'ajouter la complexité VRPTW.

NOTIONS ABORDÉES

Policy gradient

REINFORCE (Williams
1992)

Baseline variance
reduction

Descente de gradient

PyTorch autograd

Charge : 1 personne · ~2h30

Livable : Section "Algorithme RL" du notebook final

T2
1

Implémenter le modèle MLP baseline

DL

réseau de neurones

POURQUOI

Commencer par un MLP simple avant tout réseau d'attention. Si le MLP n'apprend pas, le problème vient de l'environnement ou de REINFORCE, pas du modèle. Cette approche bottom-up évite de déboguer plusieurs composants en même temps.

COMMENT

Encoder l'état en un vecteur fixe. Passer par 2-3 couches fully connected avec ReLU. Sortie = logits sur les n villes. Appliquer un masque sur les villes déjà visitées avant le softmax. Commencer avec hidden_dim=64, puis augmenter si nécessaire.

```
class PolicyMLP(nn.Module):
    def __init__(self, state_dim, n_cities, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_cities)
        )
    def forward(self, state, mask):
        logits = self.net(state)
        logits[mask] = -1e9 # masquer villes visitees
        return F.softmax(logits, dim=-1)
```

NOTIONS ABORDÉES

MLP PyTorch

Masque d'actions

ReLU activation

Softmax policy

Bottom-up debugging

Charge : 1 personne · ~1h30

Livable : Section "Modèle DL" du notebook final

T2
2

Entraîner sur instances du générateur phase 1

DL

entraînement

POURQUOI

La force du Deep Learning est de généraliser : entraîner une fois sur de nombreuses instances, puis inférer en temps constant sur n'importe quelle nouvelle instance. C'est l'argument principal pour la soutenance.

COMMENT

Générer un dataset d'entraînement avec le générateur phase 1 ($n=20$, 10 000 instances, seeds 0..9999). Entraîner le modèle sur ces instances. Valider sur 500 instances séparées (seeds 10000..10499). Mesurer le temps d'inférence sur $n=20$ et $n=100$ pour montrer $O(1)$.

```
# Entraînement
for epoch in range(n_epochs):
    instances = [generate_instance(n=20, seed=s) for s in batch_seeds]
    for instance in instances:
        loss = reinforce_step(model, instance)
    if epoch % 100 == 0: evaluate(model, val_set)

# Mesure temps inference vs n
for n in [10, 20, 50, 100, 200]:
    inst = generate_instance(n=n, seed=0)
    t = timeit(lambda: model_infer(inst), number=100) / 100
    print(f"n={n}: {t*1000:.2f} ms") # doit rester ~constant
```

Sauvegarder les checkpoints régulièrement (torch.save). Un crash sans checkpoint = recommencer depuis le début.

NOTIONS ABORDÉES

Dataset d'entraînement

Train / validation split

Inférence $O(1)$

Généralisation

timeit benchmark

Charge : 1 personne · ~2h (+ temps GPU/CPU)**Livrable** : Section "Entraînement DL" du notebook final

T2
3

Réseau d'attention (Kool 2019) si temps disponible

DL avancé

attention

POURQUOI

L'architecture d'attention (Transformer adapté au TSP) est plus puissante que le MLP car elle capture les relations entre toutes les villes. C'est la référence académique. Tenter cette implémentation donne un argument fort pour la soutenance et rapproche des résultats de la littérature.

COMMENT

Encoder chaque ville avec un embedding de dimension $d=128$. Appliquer $N=3$ couches d'attention multi-têtes ($h=8$). Décoder avec un mécanisme de pointer-attention sur les villes restantes.
Référence : arxiv 1803.08475 (Kool et al. 2019), code open source disponible.

Architecture simplifiée :

Encoder : `embedding(coords) -> N × MultiHeadAttention -> h`

Decoder : `query = h[current_city]`

`keys = h[remaining_cities]`

`scores = query · keys / sqrt(d)`

`probs = softmax(scores) * mask`

Référence : github.com/wouterkool/attention-learn-to-route

Code PyTorch open source – peut être adapté directement

Si le temps manque, documenter quand même pourquoi cette architecture serait meilleure — ça compte dans les "perspectives d'amélioration" de l'étude expérimentale.

NOTIONS ABORDÉES

Transformer / Attention

Multi-head attention

Pointer network

Kool et al. 2019

[arxiv:1803.08475](https://arxiv.org/abs/1803.08475)

Charge : Si temps disponible · ~4h

Livrable : Section "Architecture avancée" du notebook final (bonus)

T2
4

Logger courbe d'apprentissage et mesures finales

DL

livrable

POURQUOI

La courbe d'apprentissage (reward vs epochs) est la preuve visuelle que le modèle apprend. Sans elle, impossible de distinguer un modèle qui converge d'un modèle qui plafonne. C'est un argument fort pour la soutenance.

COMMENT

Logger à chaque epoch : récompense moyenne sur le batch d'entraînement, coût moyen sur le set de validation, temps d'inférence. Tracer ces courbes avec matplotlib. Mesurer et comparer les temps d'inférence DL vs SA vs NNH pour $n=10, 50, 200$.

Métriques à logger chaque epoch :

- reward_mean : moyenne des retours G sur le batch
- val_cost : coût moyen sur 500 instances de validation
- infer_ms : temps d'inférence moyen (ms)

Résultat attendu pour la soutenance :

NNH $n=200$: ~50 ms SA $n=200$: ~5 000 ms

DL $n=200$: ~5 ms -> argument $O(1)$ démontré

NOTIONS ABORDÉES

Courbe d'apprentissage

matplotlib logging

Temps d'inférence $O(1)$

Argument soutenance

Charge : 1 personne · ~1h

Livrable : Section "Résultats DL" du notebook final + soutenance

Répartition recommandée — P2 ■ P4 en parallèle

P1

Personne 1 — Phase 2 complète

NNH + 2-opt + contraintes + validation

- T1 0** NNH glouton — $O(n^2)$, tous les départs
- T1 1** 2-opt amélioration locale
- T1 2** Contraintes VRPTW dans NNH
- T1 3** Logger résultats (format standardisé)
- T1 4** Validation vs solution exacte

~6h · Livrable final partie 2

P2

Personne 2 — Phase 4 Deep Learning

Env. RL + REINFORCE + MLP + entraînement

- T1 9** Environnement RL (état, action, récompense)
- T2 0** Boucle REINFORCE
- T2 1** Modèle MLP baseline
- T2 2** Entraînement sur instances générateur
- T2 3** Réseau d'attention si temps (bonus)
- T2 4** Courbe apprentissage + mesures inférence

~8h · Livrable final étude expérimentale — démarre dès T7 validé

P3

Personne 3 — Phase 3 algo SA

Recuit simulé + pénalités + hyperparamètres

- T1 5** SA — boucle principale, critère acceptation
- T1 6** Contraintes soft via pénalités lambda
- T1 7** Grid search T■, alpha, n_iter

~5h · Démarre après T14 validé (P2 terminée)

P4

Personne 4 — Phase 3 comparaison + rédaction

Benchmark SA vs NNH + assemblage notebook

- T1 8** Comparer SA vs NNH+2opt — protocole 30 instances
- Rédiger sections notebook P2 et P3
- Préparer visualisations matplotlib comparatives

~5h · Démarre après T14 · Coordination avec P3

Point de synchronisation P2 ■ P4. Dès que T7 (générateur) est stable, P4 peut démarrer T19 sans attendre la fin de P2. P2 et P4 doivent utiliser exactement le même générateur avec les mêmes seeds pour que la comparaison en phase 5 soit loyale. Prévoir un point commun à mi-parcours pour aligner les formats de logging (T13 / T24).